University of Colombo School of Computing

Colombo, Sri Lanka

Towards a Multidimensional Composite Quality Index for AI-Generated Codebases

Research Proposal

SCS 3216 : Research Methods

S.M. Silva	Index No: 22001905
Praneesh S.	Index No: 22001591
R.M.I.M. de Silva	Index No: 22000331

Supervisor

Dr. Manjusri Wickramasinghe

April 15, 2026

List of Acronyms

BLEU Bilingual Evaluation Understudy

GPT Generative Pretrained Transformer

LLM Large Language Model

Contents

List of Acronyms **i**

1 Introduction **1**

1.1 Domain 2

1.2 Motivation 2

2 Problem Statement and Justification **4**

2.1 The Research Gap 4

2.2 Justification 5

2.2.1 The Efficiency Paradox: The Trade-off of Sustainability . . . 5

2.2.2 The Success/Maintainability Paradox: The "Add it and Forget it" Anti-pattern 5

2.2.3 The Security-Functionality Disconnect: The Alignment Tax 6

3 Research Aims, Questions, and Objectives **7**

3.1 Primary Aim 7

3.2 Research Questions 7

3.3 Research Objectives 8

4 Evaluation Methods and Data Requirements **9**

4.1 Data Requirements 9

4.1.1 Repository Scale and Composition 9

4.1.2 Established Benchmark Datasets (For Baseline Comparison) 10

4.1.3 Multi-Language Source Files 10

4.2 Evaluation Method 10

4.2.1 Comparative Metric Analysis (The Core Evaluation) 10
4.2.2 Baseline Benchmarking (Human vs. AI) 11
4.2.3 Multi-Dimensional Quality and Security Assessment 11

5 Timeline **13**

Chapter 1

Introduction

The software engineering landscape is currently undergoing a paradigm shift, transitioning from traditional human-centric development to a model of Artificial Intelligence-Human Paired Programming. This transformation is driven by the rapid integration of Large Language Models (LLMs) such as GPT-4, Claude, and GitHub Copilot into the Software Development Life Cycle (SDLC). Empirical evidence suggests that these tools significantly enhance developer productivity, with studies reporting speed increases of up to 56% and substantial improvements in throughput. However, this surge in velocity has introduced a "Success Paradox": models that achieve high scores in functional correctness—the ability to pass predefined unit tests—often produce codebases characterized by structural degradation, resource inefficiency, and critical security vulnerabilities (Liu and Li, 2024).

Current evaluation paradigms rely heavily on binary metrics such as $pass@k$ or textual similarity measures like BLEU, which were originally designed for natural language translation rather than the complex, formal semantics of source code. These metrics offer an "execution-time success" that does not guarantee "maintenance-time sustainability". As the industry moves from viewing AI as an insignificant coding assistant to a primary code generator, the role of the software engineer is evolving from an author to a reviewer and auditor. Consequently, there is an urgent requirement to move beyond isolated functional assessment toward a

multidimensional quality governance framework (Evtikhiev *et al.* 2023).

1.1 Domain

The proposed research is situated within the domain of **AI-Assisted Software Quality Engineering (AI-SQE)**. This field examines the long-term impact of LLM-generated code on the health, security, and operational costs of software systems. The research specifically focuses on the formulation of a composite evaluation framework that addresses three critical pillars of software quality:

- **Long-term Maintainability and Software Evolution**: This sub-domain investigates the "add it and forget it" anti-pattern, where AI tools encourage the rapid addition of new code blocks rather than the refactoring of existing ones. This leads to an "Execution-Maintenance gap," characterized by an eightfold increase in code duplication and a 60% decline in refactoring activity compared to the pre-AI era (GitClear, 2025).

- **Security Posture and Vulnerability Resilience**: This area addresses the "Security Gap" inherent in LLM-generated outputs. Despite being functionally correct, AI-generated code frequently reproduces common flaws from its training data, such as the MITRE CWE Top 25[1].

- **Operational Efficiency and Sustainability**: This sub-domain explores the "Efficiency Paradox," where the time saved during development is offset by increased costs during operation. State-of-the-art LLMs often consumes more energy in complex algorithmic tasks like dynamic programming or bit manipulation.

1.2 Motivation

The motivation for this research is from optimizing models for functional success often results in a neglect of security best practices and architectural coherence. Ex-

[1] Highlights the most common and impactful software weaknesses

isting benchmarks are largely "snippet-level," failing to capture the complexity of real-world software where vulnerabilities and inefficiencies arise from interactions between multiple components and cross-file dependencies. Furthermore, current evaluation tools are "feature-rich but signal-poor"; while developers have access to data on duplication, bugs, and runtime, they lack a unified signal to guide the acceptance or rejection of AI-generated snippets based on overall codebase health (Lian *et al.* 2025).

Chapter 2

Problem Statement and Justification

2.1 The Research Gap

The current landscape of AI-assisted software development is dominated by a state of "Functional Myopia," where the evaluation of Large Language Models (LLMs) is primarily restricted to their ability to solve isolated, snippet-level programming tasks. While modern models demonstrate high proficiency on benchmarks such as **HumanEval** and **MBPP**, these assessments focus almost exclusively on functional correctness—whether the code performs its intended task—while ignoring the broader quality characteristics essential for production-grade software (Liu *et al.* 2023).

A critical disconnect exists between a model's capacity to produce code that "runs" in a controlled environment and its ability to generate codebases that are sustainable, secure, and resource-efficient in the long run. Modern literature suggests that real-world software engineering involves complex inter-dependencies, architectural constraints, and multi-file contexts that current snippet-level benchmarks fail to simulate. Consequently, there is a significant research gap regarding the formulation of a validated composite metric that can capture these multidimensional quality signals to ensure long-term codebase health.

2.2 Justification

The necessity of this research is underscored by three emerging paradoxes identified across the domains of efficiency, maintainability, and security.

2.2.1 The Efficiency Paradox: The Trade-off of Sustainability

While AI tools undoubtedly accelerate the code-writing process, this speed often comes at the cost of long-term operational sustainability. LLMs are fundamentally trained to prioritize producing a correct result over an efficient one, often leading to sub-optimal algorithmic choices. This creates a "runtime gap" where AI-generated solutions frequently consume significantly more memory, computing power, than human-expert solutions, particularly in complex areas such as dynamic programming or systems-level programming. For production software running at scale, the increased infrastructure costs and environmental impact of this "power-hungry" code can eventually outweigh the one-time productivity gains achieved during the development phase

2.2.2 The Success/Maintainability Paradox: The "Add it and Forget it" Anti-pattern

High functional scores on standard benchmarks often mask a structural degradation within the codebase, a phenomenon described as the "add it and forget it" anti-pattern. Longitudinal studies of global repositories indicate that the rise of AI-assisted coding has correlated with a sharp increase in code duplication and a simultaneous decline in refactoring activity. Because LLMs primarily operate through an autoregressive, next-token prediction mechanism within fixed context windows, they often struggle to recognize opportunities for code reuse across a larger system. Instead of modifying existing logic, AI assistants tend to suggest entirely new code blocks, leading to locally correct but globally redundant solutions that compound technical debt and increase the maintenance burden over

time.

2.2.3 The Security-Functionality Disconnect: The Alignment Tax

A critical finding in recent security research is that there is no direct correlation between a model's functional performance and its security posture. Models that excel at passing unit tests frequently generate "bare-bones" code that lacks essential defensive layers, such as input sanitization or proper credential management. This creates an "alignment tax" where models optimized for utility and correctness may inadvertently reproduce or even amplify known vulnerabilities found in their training data. Furthermore, enhanced "reasoning" capabilities in newer models do not inherently translate to safer code in repository-level settings, as the complexity of maintaining security constraints across multiple files and dependencies often overwhelms current model capacities (Sabra *et al.* 2025).

Chapter 3

Research Aims, Questions, and Objectives

3.1 Primary Aim

To develop and validate a **Composite Quality Index (CQI)** that mathematically unifies **Correctness**, **Maintainability**, **Security**, and **Efficiency** into a single governance framework for AI-assisted development.

3.2 Research Questions

Based on the critical gaps identified in current literature, this research seeks to address the following questions:

- **RQ1: The Metric Integration Problem** How can disparate quality dimensions, such as execution efficiency, structural maintainability, and security risk, be normalized and integrated into a single, mathematically sound composite metric?
- **RQ2: The Reasoning-Security Paradox** To what extent do enhanced reasoning capabilities in LLMs correlate with the production of more secure and maintainable codebases, and how does this "reasoning budget" affect the overall quality index?

- **RQ3: The Context-Efficiency Gap** How does the reliance on snippet-level generation rather than "whole-repository" reasoning contribute to structural degradation and resource inefficiencies in AI-generated code?
- **RQ4: Language-Specific Variability** How significantly do the inherent characteristics of programming languages influence the weighting of the final composite score?

3.3 Research Objectives

To answer the research questions and fulfill the primary aim, the following objectives have been established:

- **To quantify structural degradation[1] patterns** by analyzing the relationship between the increase in code duplication and the simultaneous decline in refactoring activity within AI-assisted repositories.
- **To identify novel technical debt categories** unique to LLM-integrated systems, such as "prompt debt," "hallucinated dependencies," and "architectural blindness," for inclusion in the maintainability assessment.
- **To evaluate the "Efficiency Paradox"** through empirical benchmarking of runtime performance, memory usage, and energy consumption across diverse programming languages using frameworks like EffiBench and Mercury.
- **To establish a "Security-Weighted" functional metric** that integrates traditional $pass@k$ success rates with vulnerability prevalence (VR) and severity scores (CVSS).
- **To formulate and validate a unified mathematical model (CQI)** that assigns dynamic weights to these four pillars, providing developers with a single, reliable signal for AI-generated codebase health.

[1] Long-term decline in the quality and organization of a software project's architecture

Chapter 4

Evaluation Methods and Data Requirements

4.1 Data Requirements

To ensure the evaluation accurately reflects real-world complexities and avoids the pitfalls of "snippet-level" testing, the data collection must be scaled and scoped appropriately.

4.1.1 Repository Scale and Composition

- **Volume:** Target a large-scale longitudinal dataset mimicking the scope of recent empirical studies (e.g., analyzing upwards of 50 to 200 million changed lines of code over a 3-to-5-year period to capture pre-AI and post-AI trends).
- **Context Window Size:** Include repositories that test the upper limits of current LLM context windows (ranging from 8,000 to over 1 million tokens) to evaluate "architectural blindness" and cross-file dependency management.
- **Project Diversity:** Select a minimum of 15–20 distinct, actively maintained open-source projects (similar to the SWR-Bench methodology). These should range from web frameworks to systems-level libraries to ensure diverse architectural patterns.

- **Pull Request (PR) and Commit Data:** Collect thousands of manually verified PRs and commit histories. This data must include the "Code Churn" rate (code revised or deleted within two weeks of commit) to accurately measure the "add it and forget it" anti-pattern.

4.1.2 Established Benchmark Datasets (For Baseline Comparison)

- **Functional/Snippet Level:** HumanEval+ and EvalPlus (for rigorous unit-test validation).
- **Repository Level:** A.S.E (AI Code Generation Security Evaluation), SWE-bench, and RepoMasterEval.
- **Efficiency Level:** EffiBench and Mercury.

4.1.3 Multi-Language Source Files

Gather AI-generated and human-authored code across distinct paradigms: memory-unsafe (C/C++), managed/interpreted (Python, JavaScript), and strongly-typed (Java, TypeScript).

4.2 Evaluation Method

The evaluation methodology will utilize a comparative approach, validating the newly proposed metrics against established industry baselines to prove their necessity and effectiveness.

4.2.1 Comparative Metric Analysis (The Core Evaluation)

- **Traditional vs. Proposed Metrics:** Evaluate AI-generated code using standard metrics (like $Pass@k$, BLEU, and CodeBLEU) and simultaneously evaluate it using the proposed composite maintainability/security metrics.

- **The "Alignment Tax" Measurement:** Quantify the gap between functional correctness and security. Compare the model's $Pass@k$ score against its Vulnerability Rate (VR). Demonstrate statistically how high functional scores often mask high vulnerability rates or structural degradation.

- **SAST Comparison:** Run the AI-generated codebases through traditional Static Application Security Testing (SAST) tools (like CodeQL or SonarQube). Compare these baseline results against the detection rates of modern, AI-powered multi-agent review systems (like IRIS) to measure improvements in identifying complex, cross-file logic flaws and reducing false-positive rates.

4.2.2 Baseline Benchmarking (Human vs. AI)

- **Historical Codebase Evolution:** Compare post-2022 AI-assisted repository data against pre-2021 human-authored baselines. Specifically, compare the ratio of duplicated/copy-pasted code blocks against refactored code to highlight the structural shift in software engineering.

- **Efficiency Gap Benchmarking:** Profile the execution time, peak memory usage, and CPU energy consumption of AI-generated solutions. Compare these values directly against the human-expert solutions provided in frameworks like EffiBench to quantify the runtime performance deficit.

4.2.3 Multi-Dimensional Quality and Security Assessment

- **Object-Oriented Complexity Tracking:** Measure the AI-generated code against the Basili et al. metric suite. Compare the Weighted Methods per Class (WMC) and Coupling Between Objects (CBO) of the AI code against acceptable industry thresholds for human production code.

- **Dynamic Execution Verification:** Move beyond static analysis by utilizing dynamic oracles. Execute the generated code in a sandboxed environment to verify that it not only compiles but satisfies security specifications during runtime simulation (as seen in methodologies like HardSecBench).

- **Hallucination and Anomaly Detection:** Track the rate of "Phantom Dependencies" and hallucinated APIs using deterministic Abstract Syntax Tree (AST) analysis, comparing the generated package calls against valid registry databases (e.g., npm, PyPI).

Chapter 5

Timeline

The research will be conducted over a twelve-month period, structured into five sequential phases as outlined in Table 5.1.

Phase	**Activity**	**Duration**
Phase 1	Metric Formulation & Weighting Logic	Month 1–2
Phase 2	Data Collection (Repo Mining & Benchmarking)	Month 3–4
Phase 3	Implementation of Automated Scoring Tool	Month 5–7
Phase 4	Validation against Human Expert Judgements	Month 8–9
Phase 5	Final Analysis & Thesis Completion	Month 10–12

Table 5.1: Research Timeline

References

Evtikhiev, M. and E. Bogomolov and Y. Sokolov and T. Bryksin (Sept. 2023). "Out of the BLEU: How should we assess quality of the Code Generation models?" In: *Journal of Systems and Software* 203, p. 111741. ISSN: 0164-1212. DOI: 10.1016/j.jss.2023.111741. URL: http://dx.doi.org/10.1016/j.jss.2023.111741.

GitClear (2025). *AI Copilot Code Quality: 2025 Look Back at 12 Months of Data.* URL: https://www.gitclear.com/ai_assistant_code_quality_2025_research.

Lian, K. and B. Wang and L. Zhang and L. Chen and J. Wang and Z. Zhao and Y. Yang and M. Lin and H. Duan and H. Zhao and S. Liao and M. Guo and J. Quan and Y. Zhong and C. He and Z. Chen and J. Wu and H. Li and Z. Li and J. Yu and H. Li and D. Zhang (2025). *A.S.E: A Repository-Level Benchmark for Evaluating Security in AI-Generated Code.* arXiv: 2508.18106 [cs.SE]. URL: https://arxiv.org/abs/2508.18106.

Liu, J. and S. Li (Sept. 2024). "Toward Artificial Intelligence-Human Paired Programming: A Review of the Educational Applications and Research on Artificial Intelligence Code-Generation Tools". In: *Journal of Educational Computing Research* 62.5, pp. 1165–1195. ISSN: 0735-6331. DOI: 10.1177/07356331241240460. (Visited on 04/10/2026).

Liu, J. and C. S. Xia and Y. Wang and L. Zhang (2023). *Is Your Code Generated by ChatGPT Really Correct? Rigorous Evaluation of Large Language Models for Code Generation.* arXiv: 2305.01210 [cs.SE]. URL: https://arxiv.org/abs/2305.01210.

Sabra, A. and O. Schmitt and J. Tyler (2025). *Assessing the Quality and Security of AI-Generated Code: A Quantitative Analysis*. arXiv: 2508.14727 [cs.SE]. URL: https://arxiv.org/abs/2508.14727.